TravelingSalesmanProblem

MunechikaHaruki

2023 年 11 月 17 日

第 I 部

TSP

目次

第 I 部　TSP 1

1 **最適化問題とは** 2
- 1.1 線形計画問題の種類 . . . 2
  - 1.1.1 線形計画問題はどのような形か . . . 2
  - 1.1.2 最短路問題（TSP とおなじネットワーク計画問題）について述べよ . . . 2

2 **TSP とは** 4
- 2.1 TSP の定式化を行え . . . 4
- 2.2 TSP はどのような種類の問題に属するか . . . 4
- 2.3 TSP 問題の難しさについて説明せよ . . . 4

3 **遺伝的アルゴリズム (GeneticAlgorithm) による巡回セールスマン問題の一解法** 5
- 3.1 遺伝的アルゴリズムとはなにか . . . 5
- 3.2 枝交換交叉のやり方について述べよ . . . 5
- 3.3 突然変異のやり方を一つ述べよ . . . 6
- 3.4 TSP の最適化の流れを述べよ . . . 6

4 **枝交換交叉による GA での巡回セールスマン問題の実装** 7

5 **コンテストのデータを用いた結果** 12
- 5.1 手法の特徴と結果についての考察 . . . 13

1　最適化問題とは

目的関数をいくつかの制約条件の元で最適化 (最大化、最小化) する問題。

1.1　線形計画問題の種類

線形計画問題　目的関数が線形かつ、制約条件が線形線形の不等式または等式であるもの
非線形計画問題　線形計画問題から、線形の制約を外したもの
ネットワーク計画問題　ネットワークで表現できる最適化問題
整数問題　変数の中に、整数の範囲の定義域となるものがある問題

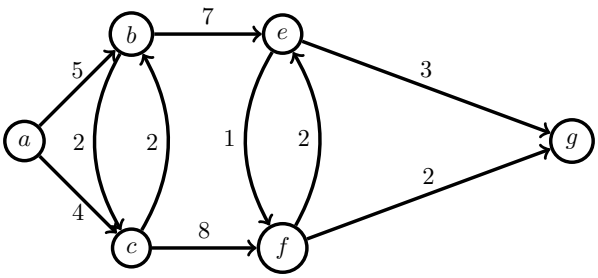
1.1.1　線形計画問題はどのような形か

線形計画問題の標準形

$$\text{目的関数}: \boldsymbol{c}^\top \boldsymbol{x} \longrightarrow \text{最小}$$
$$\text{制約条件}: \boldsymbol{A}\boldsymbol{x} = \boldsymbol{b}, \boldsymbol{x} \geq \boldsymbol{0}$$

1.1.2　最短路問題（TSP とおなじネットワーク計画問題）について述べよ

図 1: ネットワークの例



ネットワークを交通機関の路線とみなし、枝の横に書かれている数値をその路線を利用したときの所要時間とみなす。
このとき、あるノードから別のノードへの路のうち、最小の所要時間を持つ路を選ぶのが最小路問題である。
以下のように定式化できる。

0-1 条件を以下のように定義する。

0-1 条件

$$x_{ij} = \begin{cases} 1, \text{(i,j) を通る} \\ 0, \text{(i,j) を通らない} \end{cases} \tag{1}$$

路 (i,j) が持つ数値を a_{ij} とおく。すると、目的関数は、

$$\sum_{(i,j)} a_{ij} x_{ij} \tag{2}$$

と書ける。この目的関数を以下の制約条件の元で最小化するのが最短路問題である。

制約条件は、0-1 条件で、枝が出ていく数と枝が入る数がちょうどよくなるように設定する。

2　TSP とは

セールスマンが会社を出発して、いくつかの都市を訪問し、会社に戻ってくる経路の中で、最良の経路を見つける問題である。客の都市は必ず少なくとも一度は訪問しなければならない。

グラフ理論の言葉で言うと、ハミルトン閉路を求める問題である。

2.1　TSP の定式化を行え

巡回セールス問題の定式化

n 個のノードを訪問することにする。ノードの集合を $V := 0, 1, \ldots, n-1$ とする。(ただし、$n > 3$)
全ての経路の集合を $E := \{(i,j)|i,j \in V, i \neq j\}$ と定義。
経路 (i,j) の距離を $d_{i,j}$ と定義 $(d_{ij} = d_{ji})$

$$x_{ij} := \begin{cases} 1(\text{i 番目のノードから j 番目のノードを通る時}) \\ 0(\text{i 番目のノードから j 番目のノードを通らない時}) \end{cases} \tag{3}$$

$$\text{目的関数：} \sum_{(i,j)\in E} d_{ij}x_{ij} \longrightarrow min \tag{4}$$

$$\text{制約条件：} \sum_{j\in V} x_{ij} = 1, i \in V(\text{ノード}i\text{から出るのは一本}) \tag{5}$$

$$\sum_{i\in S}\sum_{j\in S} x_{ij} \leq |S| - 1, S \subset V \tag{6}$$

$$x_{ij} \in \{0,1\}, (i,j) \in E(0-1\text{条件}) \tag{7}$$

2.2　TSP はどのような種類の問題に属するか

TSP はネットワークを用いた問題であるので、ネットワーク問題である。

また、実行可能集合は x_{ij} が 0-1 変数であることから有限個の要素からなる。よって、TSP は組み合わせ計画問題 (組み合わせ最適化問題) でもある。

2.3　TSP 問題の難しさについて説明せよ

TSP 問題は、NP 完全であることが知られている。

すなわち、多項式時間アルゴリズムのみつかっていないクラス NP 問題のうち、最も難しい問題である。

すなわち、もし問題を解く多項式時間アルゴリズムが存在するならば、クラス NP の中の全ての問題にその問題を解く多項式時間アルゴリズムの存在が保証されるということである。

TSP と同じ多項式時間アルゴリズムの例として、充足性判定問題、ナップザック問題などがある。

NP 完全問題に対し多項式時間アルゴリズムを発見するのは非常に難しいので、次善のアプローチとして、近似アルゴリズムやヒューリスティクスがある。

近似アルゴリズムとは最適解に対し、ある程度の範囲の誤差を持つ近似解を保証するアルゴリズムで、ヒューリスティクス（発見的手法）とは、精度の保証はないが、少ない計算量である程度の解が得られることが経験的に知らされているアルゴリズムである。

本レポートでは、メタ・ヒューリスティクスの解法の一つである遺伝的アルゴリズムで TSP を解く。

3　遺伝的アルゴリズム (GeneticAlgorithm) による巡回セールスマン問題の一解法

本節では、TSP 問題の近似アルゴリズムの一例として、遺伝的アルゴリズム (以下 GA と略) による巡回セールスマン問題の一解法を用いる。以下に、その詳細を示す。

3.1　遺伝的アルゴリズムとはなにか

遺伝アルゴリズムの概要

GA は、自然界における生物進化のモデル化である。
個体群の中で、環境への適応度の高い個体を選択し、交叉および突然変異を起こしながら次の世代を形成していく過程を用いたメタ・ヒューリスティクスである。
TSP に対する適用以外でも、機械学習などに対する分野でも幅広い活躍を見せている。

3.2　枝交換交叉のやり方について述べよ

TSP の解の染色体への符号化

TSP の解の染色体への符号化として、ノード $V := A, B, C, D, E, F$ のとき、

$$AB \quad BC \quad CE \quad ED \quad DF \quad FA$$

のように、連続して訪問する都市の対を巡回すべきと数だけ並べた染色体を用いる。
ここでの都市の対をマクロ遺伝子、左から i 番目のマクロ遺伝子を遺伝子座 i のマクロ遺伝子と呼ぶことにする。
マクロ遺伝子 AB があったとして、A を第一コドン、B を第 2 コドンと呼称する。

生物は次世代へと子孫をつなげる時、父親と母親の遺伝子を混ぜ合わせる。これを交叉という。

この問題において交叉とは、２つの染色体の遺伝子を一部交換することで、親個体より良い個体を生成することを示す。

交叉に相当する演算には様々なものが提案されているが、今回は枝交換交叉を用いる。

以下に、枝交換交叉のアルゴリズムを示す。

交叉に相当する操作

1. 全ての個体をランダムにペアリングし、各ペアに親１、親２とラベル分けする
2. 親 1 でランダムにマクロ遺伝子を選びその遺伝子座を i_1 とする
3. 遺伝子座 i_1 のマクロ遺伝子の第一コドンと同じ第一コドンを持つ親２のマクロ遺伝子の遺伝子座を i_2 とする。
4. i_1、i_2 の第二コドンと、もう一方の親の第一コドンが等しければ、それぞれと対応付け、その遺伝子座を j_1, j_2 とする。
5. i_1, i_2 のマクロ遺伝子をを交換する i_1, i_2 の第二コドン同士が等しければ終了
6. i_1, i_2 の後に j_2, j_1 が来るように j_2, j_1 遺伝子をもってくる。
7. $j_1 \to i_1, j_2 \to i_2$ として (3) に戻る

枝交換交叉では、第二コドンの等しい枝の交換が行われるまで枝の交換が続く。

3.3 突然変異のやり方を一つ述べよ

突然変異に相当する操作

交叉が成立しなかった個体 (遺伝子の交換が行われなかった個体) に対して、突然変異を行う。
操作として、順回路の一部を逆順にする操作を行う。

1. ランダムに 2 つのマクロ演算子を選択
2. 2 つのマクロ演算子の第一コドンと第二コドンの間で染色体を分割し、染色体をさん分割する
3. 分割された染色体の中央部を逆順にする

3.4 TSP の最適化の流れを述べよ

適応と選択

ある世代から次の世代の個体が N 体生み出されたとする。この時、親と比べて、優秀な個体 (総経路が短い) や劣った個体が生まれるはずである。(総経路が長い)
この「優秀さ」の評価指標を**適応度**と呼ぶ。
順回路長 l に対し、適応度 $f(l)$ を少しの違いでも大きな差が現れるように、冪乗則を用い

$$f(l) := (\frac{n}{l})^3 \tag{8}$$

と定義する。
子世代から、適応度の高い個体を選択する。
ただ、適応度が高い順にただ選択するだけでは、遺伝的多様性が失われ、局所最適に陥る危険性があることに注意する。

以上で、交叉演算と突然変異演算を定義した。次に、これらを組み合わせて実際に最適化を行うまでの流れを示す。

最適化までの流れ

1. N 個の個体をランダムに生成する。
2. それぞれの個体の適応度を計算する
3. 新たに親となる n 個 ($n < N$) を適応度の高い順に選択する
4. n 個の個体からランダムにペアを作り、そのペアから子を新たに N 体作る
5. 確率で突然変異させる
6. (2) に移動

世代交代のは、各世代の適応度の最大値を記録し、その変化が少なくなった時に打ち止める。

4　枝交換交叉による GA での巡回セールスマン問題の実装

上の章で示されたアルゴリズムに基づき、C 言語で solve_GeneticAlgorithm 関数を実装した。それを以下に示す。

```
//*遺伝的アルゴリズム（GA） ある程度以上の最適化に弱い*//
double solve_geneticAlgorithm(int p[],double d[N][N],int n)
{
  int generationNum=10000000;//世代の数
  int eliteNum=ELITENUM;//親の数
  int numOfIndivisuals=MAXINDIVIDUAL;//世代の個体の数
  double normPathLength=NORMPATHLENGTH;
  double percentOfMutation=MUTATE;
  int Individual[MAXINDIVIDUAL][N];//個体の経路を記録
  int newIndividual[MAXINDIVIDUAL][N];//子世代の個体の経路を記録
  double Fitness[MAXINDIVIDUAL],totalPath[MAXINDIVIDUAL];//個体ごとの適応度、総経路を記録
  int suffixOfFitness[MAXINDIVIDUAL];//tatalPath に付けられた添字で、今何番目かを記録
  int MotherRank,FatherRank;//親となる個体の順位
  int Mother[N],Father[N],Child[N];//親と子の経路
  double bestPathLength=100000000;


  //初期化
  InitializeIndividuals(Individual,n,numOfIndivisuals);
  for(int k=0;k<numOfIndivisuals;k++)
    totalPath[k]=bestPathLength;

  for(int g=0;g<generationNum;g++){
  //合計、適応度の計算
  calcTotalroute(Individual,d,n,numOfIndivisuals,totalPath);
  calcFitness(totalPath,numOfIndivisuals,Fitness);
  //添字の初期化
  for(int i=0;i<numOfIndivisuals;i++){
    suffixOfFitness[i]=i;
  }

  //添字をソート
  sortSuffix(Fitness,suffixOfFitness,numOfIndivisuals);

  //現世代での最高適応度の個体を記録
  printf("generation:%d :%lf\n",g,totalPath[suffixOfFitness[0]]);
  if(totalPath[suffixOfFitness[0]]<bestPathLength){
    bestPathLength=totalPath[suffixOfFitness[0]];
  for(int i=0;i<n;i++){
    p[i]=Individual[suffixOfFitness[0]][i];
  }
  }
```

```
    //親の選択と次世代の個体の生成
    for(int i=0;i<numOfIndivisuals;i++){
      MotherRank=rand()%eliteNum;
      FatherRank=rand()%eliteNum;
      for(int j=0;j<n;j++){
        Father[j]=Individual[suffixOfFitness[FatherRank]][j];//ここなにかある
        Mother[j]=Individual[suffixOfFitness[MotherRank]][j];
      }
      createChild(Father,Mother,Child,n);
      if(rand()/RAND_MAX<percentOfMutation)mutation(Child,n);

      for(int j=0;j<n;j++){
        newIndividual[i][j]=Child[j];
      }
    }
    //世代交代
    for(int i=0;i<numOfIndivisuals;i++)
      for(int j=0;j<n;j++)
        Individual[i][j]=newIndividual[i][j];

      if(bestPathLength<normPathLength){
        save_path("mk_neighbor.path", p,n);
        break;
      }
    }
    return bestPathLength;
  }
```

本章では、この関数 solve_geneticAlgorithm について解説する。

GeneticAlgorithm ソルバ

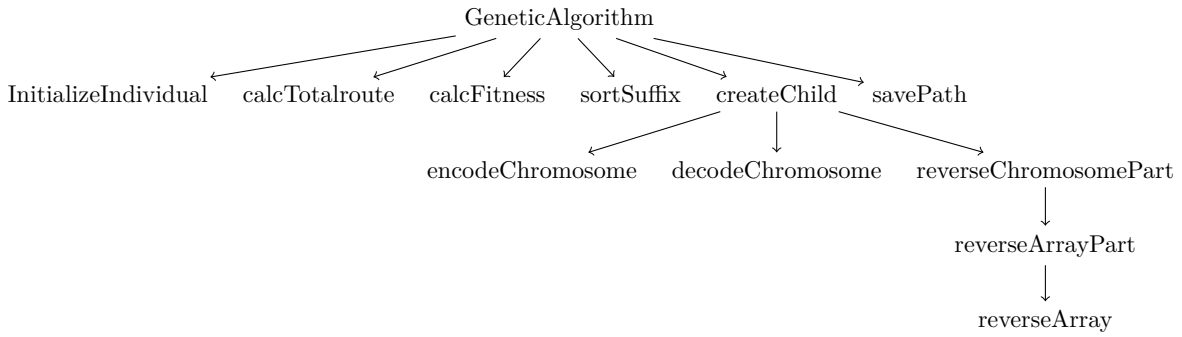
```
double solve_geneticAlgorithm(int p[],double d[N][N],int n)
```

p　　TSP の解となる経路
d　　ノード同士の距離を表す距離行列
n　　ノードの数
返り値　GA で求められた TSP の解の総経路長

ここでは `solve_geneticAlgorithm` の内部変数について解説する
() の中は変数の型を示す。

generationNum　世代の最大数。世代がこの数を超えると強制終了する (int)
eliteNum　適応度の高い順に選ばれる親の候補の数 (int)
numOfIndividuals　世代ごとの個体の数 (int)
normPathLength　経路長のノルマ。最小の経路長を持つ個体の経路長がこの数より小さくなると終わる (double)
percentOfMutation　生まれてきた子供の突然変異の確率 (double)
Individual　行に個体ごとの経路情報を格納 (int [ ][ ])
newIndividual　individual から作られた新しい世代の経路情報を格納 (int [ ][ ])
Fitness　totalPath から適応度を計算し、格納するための配列 (double [ ])
totalPath　Individual と d から個体ごとに計算された個体ごとの総経路長を格納 (double [ ])
suffixOfFitness　Individual の個体ごとに付けられる添字。totalPath と一緒に降順にソートされ、適応度の高い個体を選択する際にも使われる。(int [ ])
Mother,Father,Child　2 つの親と子個体の経路情報を格納するための配列 (int [ ])
bestPathLength　これまで作り出された個体の中での最短経路を記録 (double)

この GA ソルバは複数の関数に依存している。ここではその依存関係を上から下への階層構造として、以下に示す。



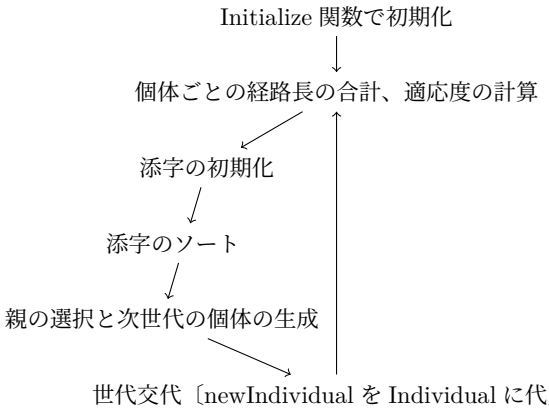
次に、上にあるような GeneticAlgorithm ソルバの内部で使われる関数について説明する。

これらの関数の実装方法については本レポートの中では解説しない。簡単な機能のみに絞る。引数と戻り地については実際のプログラム内を参照

GeneticAlgorithm ソルバの内部関数

InitializeIndividual　Individual 配列に対し、行ごとに 0 (ノード数-1) を被りなくランダムに敷き詰める関数
calcTotalroute　Individual 配列と距離行列 d から、行ごとの総経路長を計算し、配列 totalPath に格納
calcFitness　配列 totalPath から適応度を計算し結果を配列 Fitness に代入
sortSuffix　Fitness と suffixOfFitness を指定すると、Fitness が降順になり、それと同じように suffixOfFitness も入れ替える。(個体番号を適応度が大きい順にソートする関数)
createChild　2 つの親を指定することで子供を作る関数
savePath　経路を保存する関数
encodeChromosome　経路を染色体にする関数
decodeChromosome　染色体を経路にする関数
reverseChromosomePart　染色体と適当な数 i,j を指定すると、第 i 座と第 j 座の間にある全ての遺伝子を逆順にする関数
reverseArrayPart　配列を部分的に逆順にする関数
reverseArray　配列を逆順にする関数

次に、関数内での大まかな処理の流れを以下に示す。



elitefNum などのハイパラメータに代入されている N や MAXINDIVIDUAL は、オブジェクト形式マクロである。
遺伝的アルゴリズムはその性質上、ハイ・パラメータの探索が必須である。
ハイパラメータは一度に変えるのが望ましいため、オブジェクト形式マクロとしてプログラムの先頭で一気に扱うことにした。
実行時には、プログラムの頭で定義されたオブジェクト形式マクロを変化させながら、局所最適に陥らずより良い解が得られるような値を探索する。
下に、実行時にオブジェクト形式マクロ一覧を示す。

プログラム中のオブジェクト形式マクロ一覧

N　　ノードの最大数
MAXINDIVIDUAL　個体の数
NORMPATHLENGTH　ノルマとなる経路長 (これより短い経路がでてくればループが終了)
MUTATE　変異率
ELITENUM　親の数

プログラムの実行例

ハイパラメータとして、世代ごとの個体数 1000、変異率 0.8、親の数 7、総経路のノルマを 450 で与える座標データを data20 とする。このときの実行結果のうち、`solve_geneticAlgorithm` 関数による標準出力の部分を以下に示す。

```
generation:0 :1330.617481
generation:1 :1222.243377
generation:2 :1104.233158
generation:3 :994.046342
...(中略)
generation:28 :483.928012
generation:29 :481.001254
generation:30 :471.553964
generation:31 :465.178237
generation:32 :455.875895
generation:33 :455.010522
generation:34 :449.828530
```

これを見れば、`solve_geneticAlgorithm` は妥当な近似解を出力できることがわかる。

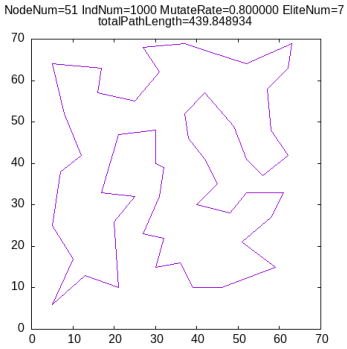
5　コンテストのデータを用いた結果

コンテストで用いた data20,data21,data22 に対し、上の GeneticAlgorithm ソルバを適用させ、近似解を得た。

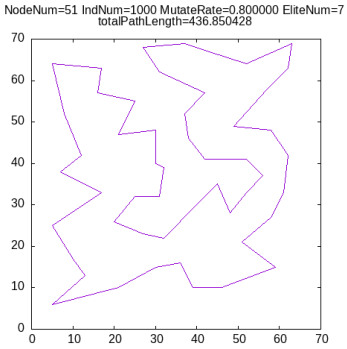
以下にそうして得られた経路の図を二通り、そしてその総経路長、またその時に使ったハイパラメータ (総経路のノルマ以外) を図のキャプションに示す。

コンテスト時に得られたデータが残っていないので、実行し直したことを追記する。

図 2: data20

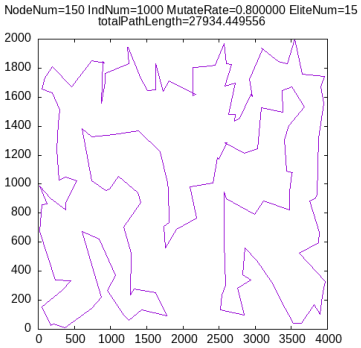


(a) 総経路長：439.848934

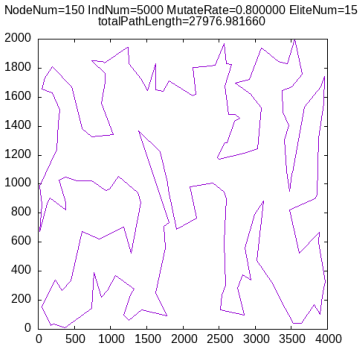


(b) 総経路長：436.850428

図 3: data21

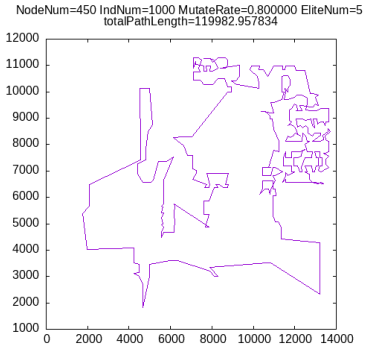


(a) 総経路長：27934.449556

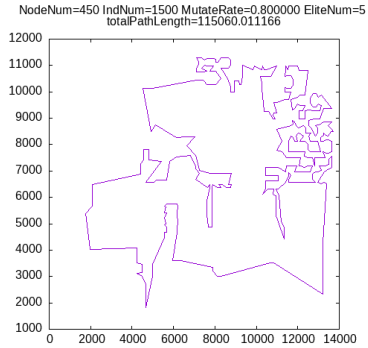


(b) 総経路長：27976.981660

図 4: data22



(a) 総経路長：119982.957834



(b) 総経路長：115060.011166

5.1 手法の特徴と結果についての考察

遺伝的アルゴリズムは、ある程度までの近似解を得ることは得意だが、最適解やほぼ限りなく最適解に近い近似解を得ることが難しいアルゴリズムである。このことは、data21 と data22 のコンテストでの順位が、90 人中 30 位程度だったことからも見て取れる。

原因として、遺伝的アルゴリズムそのものが、計算量に応じて解の収束を保証しないヒューリスティクスなアルゴリズムであることが考えられる。

それと同時に、GA は、複数のハイパラメータを設定しないといけないため、扱いが難しいアルゴリズムでもある。実際に data20 のコンテストの順位は、ハイパラメータの最適化不足でほぼ最下位になってしまった。

ハイパラメータそのものを最適化するようなアルゴリズムも考案されているが、あまり普及していない。TSP のような研究が進んでいる問題を解く際には、GA などのヒューリスティクス解法ではなく収束が保証されている近似アルゴリズムを使用するべきである。